

Familiarización con el Código Base

El proyecto está organizado bajo una **arquitectura en capas de Spring Boot**, separando modelo, persistencia, servicios y controlador. Esta estructura permite mantener **bajo acoplamiento y alta cohesión**, facilitando la extensibilidad y el mantenimiento del sistema.

Model

Estas clases representan el **dominio del sistema**, es decir, las entidades principales con las que trabaja la aplicación.

- **Blueprint:** Esta clase representa un plano arquitectónico donde este formato por los siguientes atributos
 - **Autor** -> Este se refiere al autor del plano
 - **Name** -> Este atributo hace referencia al nombre del plano
 - **List<Point> pointst** -> Lista de puntos que forman el plano

```
package edu.eci.answ.blueprints.model;

import java.util.ArrayList;
import java.util.Collections;
import java.util.List;
import java.util.Objects;

public class Blueprint {

    private String author;
    private String name;
    private final List<Point> points = new ArrayList<>();

    public Blueprint(String author, String name, List<Point> pts) {
        this.author = author;
        this.name = name;
        if (pts != null) points.addAll(pts);
    }

    public String getAuthor() { return author; }
    public String getName() { return name; }
    public List<Point> getPoints() { return Collections.unmodifiableList(points); }

    public void addPoint(Point p) { points.add(p); }

    @Override
    public boolean equals(Object o) {
        if (this == o) return true;
        if (!(o instanceof Blueprint bp)) return false;
        return Objects.equals(author, bp.author) && Objects.equals(name, bp.name);
    }

    @Override
    public int hashCode() {
        return Objects.hash(author, name);
    }
}
```

Responsabilidad

Representar la entidad central del sistema sin lógica de infraestructura.

Características

- La clase `getPoints()` evita que otras clases modifiquen directamente la lista de puntos
 - El método `addPoint(Point p)` agrega puntos
 - En `equals()` y `hashCode()` implican que un blueprint está identificado únicamente por (autor, name)
 - Point
- Esta clase usa record para:
- Inmutabilidad
 - Código compacto
 - Mayor claridad

Responsabilidad:

Representar una coordenada cartesiana (x,y).

Persistencia

Estas clases definen cómo se almacenan los blueprints

- **BlueprintPersistence:** Define el contrato que cualquier mecanismo de almacenamiento debe implementar:

```
void saveBlueprint(Blueprint bp) throws BlueprintPersistenceException;

Blueprint getBlueprint(String author, String name) throws BlueprintNotFoundException;

Set<Blueprint> getBlueprintsByAuthor(String author) throws BlueprintNotFoundException;

Set<Blueprint> getAllBlueprints();

void addPoint(String author, String name, int x, int y) throws BlueprintNotFoundException;
```

Esto permite aplicar el principio de inversión de dependencias, ya que las capas superiores dependen de la interfaz y no de una implementación concreta

- **InMemoryBlueprintPersistence:** Es la implementación concreta que almacena los blueprints en memoria.
 - Usa `ConcurrentHashMap` para permitir acceso seguro en entornos concurrentes.
 - Utiliza como clave la combinación `author:name`.
 - Incluye datos precargados para pruebas iniciales.

- Maneja excepciones personalizadas: (BlueprintPersistenceException, BlueprintNotFoundException)

Complejidad

- Guardar blueprint: $O(1)$
- Obtener blueprint específico: $O(1)$
- Obtener por autor: $O(n)$

Services

Esta clase contiene la lógica de negocio del sistema y actúa como intermediario entre el controlador y la persistencia.

- **BlueprintsServices**

```
private final BlueprintPersistence persistence;
private final BlueprintsFilter filter;
```

Esto demuestra el uso de Inyección de Dependencias (Dependency Injection), característica fundamental de Spring.

La clase usa patrón Strategy Aquí:

```
public BlueprintsServices(BlueprintPersistence persistence, BlueprintsFilter filter) {
    this.persistence = persistence;
    this.filter = filter;
}

public void addNewBlueprint(Blueprint bp) throws BlueprintPersistenceException {
    persistence.saveBlueprint(bp);
}

public Set<Blueprint> getAllBlueprints() {
    return persistence.getAllBlueprints();
}

public Set<Blueprint> getBlueprintsByAuthor(String author) throws BlueprintNotFoundException {
    return persistence.getBlueprintsByAuthor(author);
}

public Blueprint getBlueprint(String author, String name) throws BlueprintNotFoundException {
    return filter.apply(persistence.getBlueprint(author, name));
}

public void addPoint(String author, String name, int x, int y) throws BlueprintNotFoundException {
    persistence.addPoint(author, name, x, y);
}
```

1. Se obtiene el blueprint desde la persistencia.
2. Se aplica una estrategia de filtrado.
3. Se retorna el resultado.

Esto implementa el patrón Strategy, permitiendo cambiar el comportamiento del filtrado sin modificar la lógica del servicio.

Responsabilidades del servicio

- Coordinar acceso a datos.
- Aplicar reglas de negocio.
- Integrar filtros dinámicos.
- Centralizar la lógica del sistema.

Controller

El controlador es el componente que recibe una solicitud HTTP, procesa la información a través del servicio y devuelve una respuesta al cliente.

- **BlueprintsApiController:** La clase expone endpoints REST bajo la ruta `/blueprints`

Endpoints disponibles

Método HTTP	Ruta	Función
GET	<code>/blueprints</code>	Obtener todos los blueprints
GET	<code>/blueprints/{author}</code>	Obtener blueprints por autor
GET	<code>/blueprints/{author}/{bpname}</code>	Obtener blueprint específico
POST	<code>/blueprints</code>	Crear nuevo blueprint
PUT	<code>/blueprints/{author}/{bpname}/points</code>	Agregar punto